Programación Orientada a Objetos (POO) en C++: Simulador de Combate Espacial en Consola (class Nave)

Autores: Sofia Mojica, Gabriel Zacarías, Mateo Roberts y Leonel De Armas
Institución: E.E.S.T. N° 1 "Eduardo Ader" Vicente López
Materia: Laboratorio de Programación (LPR) - 5° Año 3° A-B
Docentes a cargo: Prof. York Mansilla / Prof. Yamil Ganduglia
Fecha: Octubre de 2026

---

1. Introducción

El presente informe técnico documenta el desarrollo y la implementación de la **Actividad 8**, correspondiente al segundo cuatrimestre de la materia Laboratorio de Programación (LPR). El propósito central de esta práctica consiste en dar el salto evolutivo desde las estructuras de datos tradicionales (`struct`) hacia la **Programación Orientada a Objetos (POO)** utilizando el lenguaje C++. Para ello, se desarrolló un motor de juego de naves espaciales en consola, aplicando conceptos fundamentales de la ingeniería de software como el encapsulamiento, el diseño modular mediante clases (`class`), constructores, métodos y el manejo de objetos en memoria.

---

2. Desarrollo (Cuestionario Técnico de Control)

1. `struct` vs `class`

¿Cuál es la principal diferencia técnica entre un `struct` (visto en Act. 7) y una clase, especialmente respecto a la sección `private` y `public`?
Respuesta: La diferencia técnica fundamental entre un `struct` y una `class` en C++ radica en la accesibilidad predeterminada de sus miembros (atributos y métodos). En un `struct`, todos los miembros son públicos (`public`) por defecto, lo que significa que cualquier parte del código puede acceder a ellos y modificarlos libremente. En cambio, en una `class`, los miembros son privados (`private`) por defecto. Esto obliga al programador a definir explícitamente las secciones de visibilidad, protegiendo los datos internos y estableciendo una interfaz controlada mediante métodos públicos.

2. Encapsulamiento

¿Por qué es peligroso dejar la variable `salud` como pública? ¿Qué desastre podría causar un programador novato desde el `main()`?
Respuesta: Dejar variables críticas como la salud o la energía de una nave como públicas elimina las barreras de seguridad del sistema. Si un programador novato accede directamente desde la función `main()` o desde cualquier otra parte del código, podría asignar valores ilógicos o inválidos (por ejemplo, `jugador.vidas = -999` o `jugador.salud = 500000`), corrompiendo el estado interno del objeto, rompiendo la lógica del videojuego y provocando comportamientos erráticos o cierres inesperados (bugs difíciles de depurar). El encapsulamiento previene esto al obligar a modificar el estado exclusivamente a través de métodos controlados (como `reducirVida()`) que validan las reglas del negocio.

3. Constructor

Explicá con tus palabras para qué sirve el bloque `Nave(string _nombre, ...)` dentro del código.
Respuesta: El constructor (`Nave(int _x, int _y, int _vidas)`) es un método especial que se ejecuta automáticamente en el preciso instante en que se crea o instancia un objeto de la clase `Nave`. Su función principal es inicializar los atributos internos del objeto (en este caso, las coordenadas de posición `x`, `y` y la cantidad inicial de vidas) con valores válidos y coherentes, evitando que las variables queden con "basura" en la memoria RAM y garantizando que la nave nazca lista para operar en el juego.

4. Desafío Funcionalidad

Explicá con tus palabras qué funcionalidad elegiste agregar de las 15 opciones y en qué partes del código tuviste que insertarla para que funcione.
Respuesta: Para superar el desafío obligatorio, se implementó un sistema de puntuación acumulativa y visualización de *score* en el HUD. Para integrarla, se debió añadir un atributo privado `int score` dentro de la clase correspondiente, inicializarlo en el constructor, crear un método público `incrementarScore()` y actualizar el bucle principal (`while (!gameOver)`) y el HUD en pantalla para reflejar los puntos obtenidos cada vez que una bala impacta contra un asteroide.

---

3. Conclusión

La transición de la programación estructurada basada en `struct` hacia la Programación Orientada a Objetos con `class` representa un avance indispensable en la formación técnica. A través de este simulador de combate espacial en consola, se comprobó empíricamente la utilidad del encapsulamiento para proteger la integridad de los datos y la modularidad para

estructurar código limpio, mantenible y escalable. Asimismo, el uso de herramientas de control de versiones y la documentación formal bajo estándares de la industria reafirman el perfil profesional que exige el desarrollo de software moderno.

---

4. Distribución de Tareas y Roles en la Actividad

Integrante	**Rol / Tareas Asignadas**
Sofia Mojica	Redacción de la introducción, conceptualización teórica inicial sobre el salto de `struct` a `class` y estructuración general del documento bajo formato APA v7.
Gabriel Zacarías	Desarrollo de las respuestas del cuestionario técnico, profundizando en la explicación teórica del encapsulamiento, modificadores de acceso (`private`/`public`) y constructores.
Mateo Roberts	Implementación y documentación de la sección del Desafío de Funcionalidad, detallando la lógica de programación integrada en C++ y las modificaciones del bucle principal.
Leonel De Armas	Administración de la configuración del repositorio dual-remote (GitHub), archivo `.gitignore`, licencia MIT, control de versiones y recopilación de capturas de ejecución.

---

5. Espacio de Repositorios de Integrantes

1. **Leonel De Armas:**
   Repositorio individual en GitHub para la Actividad 8:
   https://github.com/tomasdearmas99-png/ACTIVIDAD-8-PROGRAMACI-N-ORIENTADA-A-OBJETOS---LPR-5-3-A-B.git